Algorithm for file updates in Python

Project description

The content and purpose of this project is to control and identify list of addresses using python. In this project we are using methods to open, write and read files with Python. This project is used as learning purposes which is only used as imaginary organization. Focus on security purposes with python. The file is used in this project is allow_list.txt and the purpose is to address files and remove the contents.

Open the file that contains the allow list

In the first step a variable is created to store the file name as “allow_list.txt”. To open this file the method open() is used. The function open() helps to open any file stored. The keyword with helps to manage the function to specifically get the action for this code. It uses two parameters that helps when accessing files. First is the file or in this case the variable name “import_file” along with “r” argument with stands for read or read file. The name file is just the variable name we want to use, so it can be anything or any name we want.

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"
# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]
# First line of `with` statement
with open(import_file , "r") as file:
```

Read the file contents

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"
# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]
# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:
  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
  ip_addresses = file.read()
# Display `ip_addresses`
print(ip_addresses)
```

This function along with the argument “r” helps to identify that the file is reading. In this case the function read() comes in when the file needs to be read. In this code the method is applied and in this case is stored in a variable “ip_addresses”. The file name is “file” so to get access to only read the file “file.read()” is needed.

Convert the string into a list

The `.split()` function is called by appending it to a string variable. Our text is stored and saved in string. So the function .split() helps to convert the string in the text to put into a list. This appends the content(ip_addressess) into [].

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"
# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]
# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:
  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
  ip_addresses = file.read()
# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()
# Display `ip_addresses`
print(ip_addresses)
```

Iterate through the remove list

A key part of my algorithm involves iterating through the IP addresses that are elements in the `remove_list`. To do this, I incorporated a `for` loop.

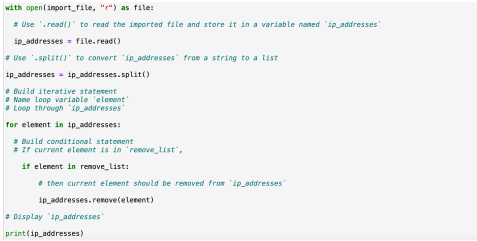
```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"
# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]
# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:
  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
  ip_addresses = file.read()
# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()
# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`
for element in ip_addresses:

    # Display `element` in every iteration
    print(element)
```

A loop repeats a code for a specific sequence. This is why the loop is needed to iterate each element in the list that is now stored in the list. The “for” starts the loop and “in” helps to identify

the content or element if in inside the loop.

Remove IP addresses that are on the remove list

The purpose of this project is to remove the Ip_addresses that are in the list. Stored as remove_list since those Ips are needed to be removed. Within the for loop, a conditional statement is created to check and compare if the contents or elements of the remove_llist are in the file or current list. If this condition meets then it will execute and remove the element.



```
with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

  ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

  # Build conditional statement
  # If current element is in `remove_list`,

    if element in remove_list:

        # then current element should be removed from `ip_addresses`

        ip_addresses.remove(element)

# Display `ip_addresses`

print(ip_addresses)
```

Update the file with the revised list of IP addresses

To update the list file. We need the .join() method that combines all items in an iterable into a string. the `.join()` method to create a string from the list `ip_addresses` so that I could pass it in as an argument to the `.write()` method when writing to the file `"allow_list.txt"`. I used the string `("\n")` as the separator to instruct Python to place each element on a new line. The another with statement is created to write into the file. So, in this case is similar to the read method but since we are writing into a file we need to “w” and the .write() method to update the file. This argument indicates that I want to open a file to write over its contents. When using this argument `"w"`, I can call the `.write()` function in the body of the `with` statement.
The `.write()` function writes string data to a specified file and replaces any existing file content.

```
for element in ip_addresses:

  # Build conditional statement
  # If current element is in `remove_list`,

    if element in remove_list:

        # then current element should be removed from `ip_addresses`

        ip_addresses.remove(element)

# Convert `ip_addresses` back to a string so that it can be written into the text file

ip_addresses = " ".join(ip_addresses)

# Build `with` statement to rewrite the original file

with open(import_file , "w") as file:

  # Rewrite the file, replacing its contents with `ip_addresses`

  file.write(ip_addresses)
```

Summary

This project is created to understand how to use Python in cyber security along with the understanding of developing skills using files. The purpose of this code is to open , write and remove content of file and in this case IPS. Different methods are used in this code as .split(), .join() and remove() which are necessary part of the function of this code.